

```
1 =====
2 NETWORK MARVELS – SOFTWARE ENGINEER (ENGINEER LEVEL QUESTIONS)
3 Puzzles + Deep STL + Vector Internals + String Internals
4 NOT in your previous prep docs.
5 For: Samaranjan Manjhi | Interview: 16 June 2026, 2 PM
6 =====
7 Engineer-level = they expect you to know WHY, not just WHAT.
8 They want internal mechanics, edge cases, and real trade-offs.
9 =====
10
11
12 =====
13 SECTION 1 – CLASSIC ENGINEER PUZZLES
14 (These are asked to test logical thinking and problem decomposition)
15 =====
16
17 -----
18 PUZZLE 1: 8 Balls, 1 Odd (lighter) – 2 Weighings
19 -----
20 Problem: 8 balls, one is lighter. Balance scale, 2 weighings. Find it.
21
22 ANSWER:
23 Weighing 1: Put 3 balls on left, 3 on right.
24     Case A – one side lighter → odd ball is in that group of 3.
25     Case B – balanced → odd ball is in remaining 2.
26
27 Weighing 2:
28     Case A: Take 2 balls from lighter group. Weigh them.
29         If one side lighter → that's it.
30         If balanced → third ball (not weighed) is the odd one.
31     Case B: Weigh the remaining 2 against each other.
32         Lighter one is odd.
33
34 ENGINEER FOLLOW-UP: "What if you don't know if it's lighter or heavier?"
35 ANSWER: Need 3 weighings. Label balls 1-8. First weigh 1,2,3 vs 4,5,6.
36 Track which pan dips – now you know heavier OR lighter. Then narrow down.
37
38 -----
39 PUZZLE 2: 2 Eggs, 100 Floors – Minimum Worst Case Drops
40 -----
41 Problem: 2 eggs, 100-floor building. Find the highest floor an egg
42 can be dropped from without breaking. Minimize worst-case drops.
43
44 ANSWER (the engineer answer, not the naive one):
45 Naive wrong answer: Binary search =  $O(\log n)$  but only works with
46 unlimited eggs. With 2 eggs, if first egg breaks, you must do linear
47 search from below – so binary search doesn't work.
48
49 Correct approach: Try floors 14, 27, 39, 50, 60, 69, 77, 84, 90, 95, 99, 100.
50 Gap decreasing by 1 each time (14, 13, 12, 11...).
51 Worst case: 14 drops.
```

52

53 Formula: $k + (k-1) + (k-2) + \dots + 1 \geq 100 \rightarrow k(k+1)/2 \geq 100 \rightarrow k=14$.

54

55 WHY DECREASING GAPS: As egg 1 is dropped more times, egg 2 has fewer

56 floors left to linearly scan. Keeps worst case flat.

57

58 -----

59 PUZZLE 3: Pirates and Gold (Logic)

60 -----

61 Problem: 5 pirates rank 1-5 (1=most senior). They found 100 gold coins.

62 Senior pirate proposes split. All vote. If $\geq 50\%$ agree, done.

63 If not, proposer thrown overboard and next senior proposes.

64 Pirates are rational and greedy. What does Pirate 1 propose?

65

66 ANSWER: Work backwards.

67 2 pirates: Pirate 2 takes all 100. Pirate 1 gets 0 (loses vote).

68 3 pirates: Pirate 3 needs 2 votes. Offers pirate 1 just 1 coin

69 (pirate 1 gets 0 if 3 dies, so takes 1). Pirate 3: 99, P2: 0, P1: 1.

70 4 pirates: Pirate 4 needs 2 votes. Offers P2 1 coin (P2 gets 0 if 4 dies).

71 P4: 99, P3: 0, P2: 1, P1: 0.

72 5 pirates: Pirate 5 needs 3 votes. Offers P3 and P1 each 1 coin.

73 Final: P5=98, P4=0, P3=1, P2=0, P1=1.

74

75 WHAT THIS TESTS: Recursive reasoning, backward induction – same thinking

76 used in game trees (minimax). Relevant to algorithm design.

77

78 -----

79 PUZZLE 4: Odd One Out (Bit Trick – CODE)

80 -----

81 Problem: Array has n elements where every element appears twice

82 except one. Find the odd one out. $O(n)$ time, $O(1)$ space.

83

84 ANSWER:

85 `int findOdd(vector<int>& arr) {`

86 `int res = 0;`

87 `for(int x : arr) res ^= x;`

88 `return res;`

89 `}`

90 `// XOR property:  $a^a=0$ ,  $a^0=a$ , XOR is commutative and associative.`

91 `// All pairs cancel to 0. Only unpaired element remains.`

92

93 FOLLOW-UP: What if two elements appear odd number of times?

94 `int x = 0;`

95 `for(int n : arr) x ^= n; //  $x = a^b$  (XOR of the two odd elements)`

96 `int bit = x & (-x); // rightmost set bit (differs between a and b)`

97 `int a = 0;`

98 `for(int n : arr) if(n & bit) a ^= n; // partition and XOR`

99 `int b = x ^ a;`

100 `// Result: a and b.`

101

102 -----

```

103 PUZZLE 5: Water Jug Problem (Classic)
104 -----
105 Problem: 3L jug and 5L jug, unlimited water. Measure exactly 4L.
106
107 ANSWER:
108 Step 1: Fill 5L jug.
109 Step 2: Pour from 5L into 3L → 5L has 2L, 3L is full.
110 Step 3: Empty 3L.
111 Step 4: Pour 2L from 5L into 3L.
112 Step 5: Fill 5L again.
113 Step 6: Pour from 5L into 3L (needs 1L to fill) → 5L now has 4L.
114
115 ENGINEER NOTE: This is BFS in disguise. State = (x, y) where x=3L jug,
116 y=5L jug. BFS from (0,0) to any state where x=4 or y=4.
117
118 -----
119 PUZZLE 6: Josephus Problem (CODE – engineers love this)
120 -----
121 Problem: n people in circle, every k-th person eliminated. Who survives?
122 Classic case: n people, eliminate every 2nd. Find survivor's position.
123
124 ANSWER (0-indexed):
125 int josephus(int n) {
126     // k=2 special case – O(log n) solution
127     // Find highest power of 2 ≤ n, then: survivor = 2*(n - 2^L) + 1
128     int p = 1;
129     while(p * 2 ≤ n) p *= 2;
130     return 2 * (n - p) + 1;
131 }
132 // n=6: p=4, survivor = 2*(6-4)+1 = 5
133
134 General k (O(n)):
135 int josephus(int n, int k) {
136     int pos = 0; // survivor's position in 1-person circle
137     for(int i = 2; i ≤ n; i++)
138         pos = (pos + k) % i;
139     return pos + 1; // 1-indexed
140 }
141
142 WHY THIS IS ASKED: Tests modular arithmetic + recurrence relations.
143
144 -----
145 PUZZLE 7: Find Missing and Duplicate in Array (One Pass)
146 -----
147 Problem: Array 1..n with one missing number and one duplicate. Find both.
148
149 ANSWER:
150 pair<int,int> findMissingDup(vector<int>& arr) {
151     int n = arr.size();
152     long long sumExpected = (long long)n*(n+1)/2;
153     long long sum2Expected = (long long)n*(n+1)*(2*n+1)/6;

```

```
154     long long sum=0, sum2=0;
155     for(int x : arr) { sum += x; sum2 += (long long)x*x; }
156     // sum = sumExpected - missing + dup
157     // sum2 = sum2Expected - missing^2 + dup^2
158     long long diff = sumExpected - sum;           // missing - dup
159     long long diff2 = sum2Expected - sum2;         // missing^2 - dup^2
160     // diff2 = (missing+dup)(missing-dup) = (missing+dup)*diff
161     long long sumMD = diff2 / diff;               // missing + dup
162     int missing = (diff + sumMD) / 2;
163     int dup = (sumMD - diff) / 2;
164     return {missing, dup};
165 }
```

166 Time: O(n), Space: O(1)

167 -----

168 PUZZLE 8: Clock Angle Problem

169 -----

170 Problem: What is the angle between hour and minute hands at 3:27?

171 ANSWER:

172 Minute hand: $27 * 6 = 162$ degrees (moves 6° per minute)

173 Hour hand: $3*30 + 27*0.5 = 90 + 13.5 = 103.5$ degrees

174 (moves 30° per hour = 0.5° per minute)

175 Angle between = $|162 - 103.5| = 58.5$ degrees

176 FORMULA: $|30H - 5.5M|$ where H=hours, M=minutes.

177 = $|30*3 - 5.5*27| = |90 - 148.5| = 58.5$ degrees ✓

178 -----

179 PUZZLE 9: 100 Lockers Problem

180 -----

181 Problem: 100 lockers, all closed. 100 people. Person k toggles every

182 k-th locker. Which lockers are open at the end?

183 ANSWER: Lockers with an ODD number of factors – i.e., PERFECT SQUARES.

184 Open lockers: 1, 4, 9, 16, 25, 36, 49, 64, 81, 100. (10 lockers)

185 WHY: Locker n is toggled once per divisor. Most numbers have divisors

186 in pairs (d and n/d). Perfect squares have one divisor that pairs with

187 itself (sqrt). So only perfect squares are toggled odd times → left open.

188 WHAT THIS TESTS: Number theory reasoning, not trial and error.

189 =====

190 SECTION 2 – DEEP VECTOR INTERNALS (Engineer Level)

191 =====

192 "Engineer" means: you know WHY push_back is amortized O(1),

193 not just THAT it is.

194 -----

Q. How does `std::vector` manage memory internally?

ANSWER: vector maintains three pointers internally:

`_begin`: start of allocated memory

`_end`: one past last element (size)

`_cap_end`: one past allocated capacity

When `push_back` is called and `size==capacity`:

1. Allocate new block (typically 2x current capacity)
2. Move (or copy if no move ctor) all elements to new block
3. Destroy old elements + deallocate old block
4. Insert new element

WHY 2x (doubling): Amortized analysis. If you start with capacity 1 and push n elements, total copies = $1+2+4+\dots+n = 2n$. Cost per `push_back` = $O(1)$ amortized.

TRICK QUESTION: "Is it always 2x?"

ANSWER: No – the standard says "geometrically" but doesn't require 2x.

GCC uses 2x. MSVC uses 1.5x (less memory waste, slightly more copies).

Q. What is the difference between `size()` and `capacity()`?

What does `reserve()` do vs `resize()`?

ANSWER:

`size()`: number of elements currently in vector

`capacity()`: total allocated slots ($\geq$ size)

`reserve(n)`: ensures capacity $\geq n$, NO new elements created, size unchanged

`resize(n)`: sets size to n . If $n >$ current size, new elements value-initialized (0 for int, default-constructed for objects).

If $n <$ current size, elements after n are destroyed.

CODE TRAP:

```
vector<int> v;
```

```
v.reserve(10); // capacity=10, size=0
```

```
v.resize(10); // capacity=10, size=10, all elements = 0
```

```
v[5] = 42; // safe after resize, NOT safe after reserve
```

Q. When do vector iterators get invalidated?

(Engineer-level – they will ask this for your daemon code)

ANSWER (complete rules):

1. `push_back` / `emplace_back`: ALL iterators invalidated IF reallocation occurs (`size == capacity`). If no reallocation, only `end()` is invalidated.
2. insert in middle: ALL iterators at/after insertion point invalidated. If reallocation: ALL iterators invalidated.
3. `erase`: iterators at/after erased position invalidated.
4. `resize`: same as `push_back` rules.

```

256 5. clear(): all iterators invalidated (but capacity unchanged).
257 6. reserve(): if reallocation occurs, ALL invalidated.
258
259 SAFE PATTERN for erase-while-iterate:
260 // WRONG:
261 for(auto it=v.begin(); it!=v.end(); ++it)
262     if(*it == target) v.erase(it); // it now invalid!
263
264 // CORRECT (erase-remove idiom):
265 v.erase(remove(v.begin(), v.end(), target), v.end());
266
267 // OR:
268 auto it = v.begin();
269 while(it != v.end()) {
270     if(*it == target) it = v.erase(it); // erase returns next valid it
271     else ++it;
272 }
273
274 -----
275 Q. What is the erase-remove idiom? Why does it exist?
276 -----
277 ANSWER: std::remove doesn't actually remove – it moves "kept" elements
278 to the front and returns an iterator to the new end. Actual removal
279 needs a second call to erase.
280
281 vector<int> v = {1, 2, 3, 2, 4, 2};
282 v.erase(remove(v.begin(), v.end(), 2), v.end());
283 // v = {1, 3, 4}
284
285 WHY: remove is an algorithm – it works on any range/iterator pair.
286 It can't call erase because it doesn't know the container.
287 erase is a container method. Two-step by design.
288
289 YOUR LINK: "In my DLP scanner's file list, I use erase-remove to clean
290 up entries that no longer exist on disk after a scan cycle."
291
292 -----
293 Q. vector<bool> – what is special about it and why is it dangerous?
294 -----
295 ANSWER: vector<bool> is a template specialization that packs bits.
296 Each bool stored as 1 bit, not 1 byte. Saves memory.
297
298 PROBLEM: operator[] returns a proxy object, not a reference to bool.
299 So this breaks:
300 bool& ref = v[0]; // ERROR – can't take reference to a bit
301 auto x = v[0];    // x is proxy object, not bool – surprises possible
302
303 ALSO: vector<bool> is not a contiguous array of bools. Passing
304 &v[0] to a C API expecting bool* is undefined behavior.
305
306 USE INSTEAD: vector<char>, vector<uint8_t>, or std::bitset (fixed size).

```

```

307 -----
308
309 Q. What is std::vector::shrink_to_fit()? Is it guaranteed?
310 -----
311 ANSWER: shrink_to_fit() is a non-binding request to reduce capacity
312 to match size. The implementation MAY ignore it.
313 Use case: after many erases, you want to reclaim memory.
314
315 v.resize(10);      // size=10, capacity=1000 (was large)
316 v.shrink_to_fit(); // requests capacity=10, not guaranteed
317
318 SWAP TRICK (pre-C++11 guaranteed shrink):
319 vector<int>(v).swap(v);
320 // Creates temp copy (capacity=size), swaps with v, temp (old large v) destroyed.
321
322 -----
323 Q. emplace_back vs push_back – when does it actually matter?
324 -----
325 ANSWER: For primitive types (int, double): no difference.
326 For complex objects: push_back requires either copy or move from a
327 temporary. emplace_back forwards constructor args directly – NO temp.
328
329 struct Point { int x, y; Point(int x, int y) : x(x), y(y) {} };
330
331 v.push_back(Point(1, 2)); // construct Point, then move it into vector
332 v.emplace_back(1, 2);    // construct Point directly in vector – zero copies
333
334 ALSO: push_back requires an existing object. emplace_back can handle
335 types without copy/move constructors (if placed correctly).
336
337 -----
338 Q. Write a function to remove duplicates from a sorted vector in-place.
339 Return new size. O(n) time, O(1) space.
340 -----
341 int removeDuplicates(vector<int>& v) {
342     if(v.empty()) return 0;
343     int write = 1;
344     for(int i = 1; i < v.size(); i++)
345         if(v[i] != v[i-1]) v[write++] = v[i];
346     v.resize(write);
347     return write;
348 }
349 // {1,1,2,3,3,4} → {1,2,3,4}, returns 4
350
351 FOLLOW-UP: Unsorted vector, preserve order, O(n)?
352 int removeDupsUnsorted(vector<int>& v) {
353     unordered_set<int> seen;
354     int write = 0;
355     for(int x : v)
356         if(seen.insert(x).second) v[write++] = x;
357     v.resize(write);

```

```

358     return write;
359 }
360
361 -----
362 Q. What is std::span (C++20) and how would you use it?
363     (Forward-thinking engineer question)
364 -----
365 ANSWER: std::span is a non-owning view over contiguous memory –
366 an array, vector, or C-style array. Like a (pointer, size) pair
367 but type-safe and with STL iterator support.
368
369 void process(std::span<const int> data) {
370     for(int x : data) { /* works on any contiguous range */ }
371 }
372
373 vector<int> v = {1,2,3};
374 int arr[] = {4,5,6};
375 process(v);    // works
376 process(arr);  // works
377 process({v.data()+1, 2}); // works – subspan
378
379 WHY IT MATTERS FOR YOUR WORK: "In my kernel verifier, I pass raw
380 buffer + length everywhere. std::span would replace that with
381 type-safe, bounds-aware views – no accidental out-of-bounds."
382
383
384 =====
385 SECTION 3 – DEEP STRING INTERNALS (Engineer Level)
386 =====
387
388 -----
389 Q. How is std::string implemented? What is SSO?
390 -----
391 ANSWER: std::string typically stores:
392 - A pointer to heap-allocated char buffer
393 - size (current length)
394 - capacity (allocated bytes)
395
396 SSO = Small String Optimization. For strings <= ~15 chars (varies by
397 implementation), the characters are stored INSIDE the string object
398 itself – no heap allocation. The same bytes used for the pointer are
399 used directly as the buffer.
400
401 WHY IT MATTERS: Short strings (file extensions, field names, flags)
402 have zero allocation cost. Profiler won't show them as heap hotspots.
403
404 TRAP: SSO means sizeof(std::string) is typically 24-32 bytes regardless
405 of string content. Don't assume small string = small object.
406
407 -----
408 Q. std::string vs std::string_view – when to use which?

```



```

409 -----
410 ANSWER:
411 string: owns the character data. Heap allocated (unless SSO). Can modify.
412 string_view: non-owning view into existing string/char array. No copy.
413         Pointer + length only. Cannot modify. Cannot outlive source.
414
415 USE string_view for: read-only function parameters, parsing substrings.
416 USE string for: when you need ownership or modification.
417
418 // WRONG – dangling string_view:
419 string_view sv = string("temp"); // string destroyed, sv dangling!
420
421 // RIGHT:
422 string s = "hello world";
423 string_view sv = s;           // sv points into s – safe as long as s lives
424 string_view word = sv.substr(6, 5); // "world" – no copy
425
426 YOUR LINK: "In my PCRE2 DLP engine, I pass string_view to the pattern
427 matcher – avoids copying the file content into a new string for each
428 regex check."
429
430 -----
431 Q. What is the difference between find() and rfind() on strings?
432     What does find() return on failure?
433 -----
434 ANSWER:
435 find(sub, pos=0): first occurrence of sub at or after pos
436 rfind(sub, pos=npos): last occurrence of sub at or before pos
437
438 CRITICAL: On failure, both return string::npos (= SIZE_MAX = a very large
439 unsigned value). Common bug:
440
441 // WRONG:
442 if(s.find("abc") >= 0) { ... } // ALWAYS true – npos is unsigned, >= 0 always
443
444 // CORRECT:
445 if(s.find("abc") != string::npos) { ... }
446
447 -----
448 Q. Implement strstr() – find substring in string without using find().
449 -----
450 int strstrCustom(const string& haystack, const string& needle) {
451     if(needle.empty()) return 0;
452     int n=haystack.size(), m=needle.size();
453     for(int i=0; i<=n-m; i++) {
454         int j=0;
455         while(j<m && haystack[i+j]==needle[j]) j++;
456         if(j==m) return i;
457     }
458     return -1;
459 }

```

```

460 Time:  $O(n*m)$  naive.  $O(n)$  with KMP.
461
462 KMP (they may ask this for engineer level):
463 void buildLPS(const string& p, vector<int>& lps) {
464     int m=p.size(), len=0, i=1;
465     lps[0]=0;
466     while(i<m) {
467         if(p[i]==p[len]) lps[i++]=++len;
468         else if(len) len=lps[len-1];
469         else lps[i++]=0;
470     }
471 }
472 int KMPSearch(const string& text, const string& pat) {
473     int n=text.size(), m=pat.size();
474     vector<int> lps(m);
475     buildLPS(pat, lps);
476     int i=0, j=0;
477     while(i<n) {
478         if(text[i]==pat[j]) { i++; j++; }
479         if(j==m) return i-j; // found
480         else if(i<n && text[i]!=pat[j]) {
481             if(j) j=lps[j-1];
482             else i++;
483         }
484     }
485     return -1;
486 }
487 Time:  $O(n+m)$ 
488
489 WHY ENGINEER CARES: KMP was used in your PCRE2 engine for literal
490 pattern fast-path before regex match. Same principle.
491
492 -----
493 Q. Implement a string split function (not in standard C++17 and below).
494 -----
495 vector<string> split(const string& s, char delim) {
496     vector<string> res;
497     stringstream ss(s);
498     string token;
499     while(getline(ss, token, delim))
500         if(!token.empty()) res.push_back(token);
501     return res;
502 }
503
504 // OR without stringstream (faster, no copy):
505 vector<string_view> splitView(string_view s, char delim) {
506     vector<string_view> res;
507     size_t start=0, end;
508     while((end=s.find(delim, start)) != string_view::npos) {
509         if(end>start) res.push_back(s.substr(start, end-start));
510         start=end+1;

```

```
511     }
512     if(start<s.size()) res.push_back(s.substr(start));
513     return res;
514 }
```

515

516 YOUR LINK: "I use split on config file lines in my FIM daemon – each
517 line is 'path:event_mask'. string_view variant avoids copying paths."

518

519 -----

520 Q. What is the complexity of string concatenation in a loop?
521 How do you fix it?

522 -----

523 ANSWER: s += piece is O(n) per step (may reallocate). Over k pieces
524 of average length m: O(k*m) amortized due to doubling. But naive + in
525 a loop:

```
526 string result = "";
527 for(string& s : pieces) result = result + s; // O(n²) – creates new string each time!
```

528

529 FIX 1: Use +=

```
530 for(string& s : pieces) result += s; // O(n) amortized
```

531

532 FIX 2: Use stringstream for many pieces:

```
533 ostringstream oss;
534 for(string& s : pieces) oss << s;
535 string result = oss.str();
```

536

537 FIX 3: Pre-calculate total length, reserve, then append:

```
538 size_t total = 0;
539 for(auto& s : pieces) total += s.size();
540 result.reserve(total);
541 for(auto& s : pieces) result += s;
```

542

543 YOUR LINK: "In my JSON log builder for the ransomware module, I build
544 log strings this way – reserve the estimated size first."

545

546 -----

547 Q. What is the output? Explain.

```
548     string s = "hello";
549     s[0] = 'H';
550     const char* p = s.c_str();
551     s += " world";
552     printf("%s\n", p);
```

553 -----

554 ANSWER: UNDEFINED BEHAVIOR.

555 s += " world" may reallocate the internal buffer. p still points to
556 the OLD buffer which is now freed/invalid. Accessing p is UB.

557

558 RULE: After any modifying operation on string, all pointers obtained
559 from c_str() or data() are invalidated.

560

561 -----

```

562 Q. Implement atoi (string to int) handling edge cases.
563     (Different from your PDF – this version asks for edge cases verbally)
564 -----
565 Edge cases to mention:
566 1. Leading whitespace – skip it
567 2. Optional '+' or '-' sign
568 3. Non-digit characters after digits – stop there (don't return error)
569 4. Overflow – clamp to INT_MAX or INT_MIN
570 5. Empty string / only whitespace → return 0
571 6. " -2147483649" → clamp to INT_MIN
572
573 int myAtoi(string s) {
574     int i=0, sign=1;
575     long result=0;
576     while(i<s.size() && s[i]==' ') i++;
577     if(i<s.size() && (s[i]=='+'||s[i]=='-'))
578         sign=(s[i++]=='+') ? 1 : -1;
579     while(i<s.size() && isdigit(s[i])) {
580         result=result*10+(s[i++]-'0');
581         if(result*sign > INT_MAX) return INT_MAX;
582         if(result*sign < INT_MIN) return INT_MIN;
583     }
584     return (int)(result*sign);
585 }
586
587
588 =====
589 SECTION 4 – DEEP STL INTERNALS (Engineer Level)
590 =====
591
592 -----
593 Q. How does unordered_map work internally? What is a hash collision?
594     How is it resolved in the STL?
595 -----
596 ANSWER:
597 Internally: array of "buckets". Key → hash function → bucket index.
598 Each bucket stores a linked list (chaining) of key-value pairs.
599
600 Collision: two keys hash to same bucket. Both go into the same list.
601 Lookup: hash → bucket → linear search the list.
602
603 Load factor = size / bucket_count.
604 When load factor > max_load_factor (default 1.0): rehash – double
605 buckets, reinsert all elements. O(n) rehash, amortized O(1) per insert.
606
607 WORST CASE: All keys collide → one huge list → O(n) lookup.
608 This can be triggered deliberately (hash DoS attack) with bad hash functions.
609
610 YOUR LINK: "In my PCRE2 match cache I use unordered_map with a custom
611 hash to avoid collisions on file paths – file paths often share prefixes
612 which naive hash functions handle poorly."

```

```

613 -----
614
615 Q. What is the difference between map and unordered_map for iteration order?
616 -----
617 ANSWER:
618 map: stores in sorted order by key (Red-Black tree). Iteration in
619 ascending key order always. Predictable.
620
621 unordered_map: no guaranteed order. Order depends on hash and bucket
622 layout. May change after rehash.
623
624 USE map when: you need sorted iteration, range queries (lower_bound,
625 upper_bound), or the key type has no hash.
626 USE unordered_map when: you only need lookup by exact key,  $O(1)$  vs  $O(\log n)$ .
627
628 ENGINEER TRAP:
629 unordered_map<string,int> m;
630 // After: m["c"]=3; m["a"]=1; m["b"]=2;
631 // Iterating m does NOT give a,b,c – order is undefined.
632
633 -----
634 Q. What is std::lower_bound vs std::upper_bound? Implement binary search
635 using them.
636 -----
637 ANSWER:
638 lower_bound(begin, end, val): first iterator where *it >= val
639 upper_bound(begin, end, val): first iterator where *it > val
640
641 // Count occurrences of val in sorted vector:
642 int count(vector<int>& v, int val) {
643     auto lo = lower_bound(v.begin(), v.end(), val);
644     auto hi = upper_bound(v.begin(), v.end(), val);
645     return hi - lo;
646 }
647
648 // Check if val exists:
649 bool exists(vector<int>& v, int val) {
650     auto it = lower_bound(v.begin(), v.end(), val);
651     return it != v.end() && *it == val;
652 }
653
654 // Insert maintaining sort:
655 void sortedInsert(vector<int>& v, int val) {
656     v.insert(lower_bound(v.begin(), v.end(), val), val);
657 }
658
659 YOUR LINK: "In my signature database, I use lower_bound to quickly
660 check if a binary path exists in the sorted key list before hitting SQLite."
661
662 -----
663 Q. What is the complexity of std::set operations? How is it implemented?

```

```

664 -----
665 ANSWER: std::set uses a Red-Black Tree (self-balancing BST).
666 insert:  $O(\log n)$  – tree traversal + rebalance
667 find:  $O(\log n)$ 
668 erase:  $O(\log n)$ 
669 begin/end:  $O(1)$  (cached min/max pointers)
670 Iteration:  $O(n)$  – in-order traversal of BST → sorted output
671
672 COMPARE unordered_set:
673 insert:  $O(1)$  avg,  $O(n)$  worst
674 find:  $O(1)$  avg,  $O(n)$  worst
675 No ordering, no lower_bound/upper_bound.
676
677 USE set when: need sorted iteration, range queries, or keys not hashable.
678 USE unordered_set when: only need  $O(1)$  lookup, don't need order.
679
680 -----
681 Q. What is priority_queue? How to make a min-heap?
682 -----
683 ANSWER: priority_queue is a max-heap by default (largest at top).
684
685 // Max-heap (default):
686 priority_queue<int> maxPQ;
687 maxPQ.push(3); maxPQ.push(1); maxPQ.push(4);
688 maxPQ.top(); // 4
689
690 // Min-heap:
691 priority_queue<int, vector<int>, greater<int>> minPQ;
692 minPQ.top(); // smallest
693
694 // Custom comparator (e.g., sort by second element of pair):
695 auto cmp = [](pair<int,int>& a, pair<int,int>& b) {
696     return a.second > b.second; // min-heap by second
697 };
698 priority_queue<pair<int,int>, vector<pair<int,int>>, decltype(cmp)> pq(cmp);
699
700 ENGINEER USE CASE:
701 "In my event prioritizer for the fanotify daemon, high-severity events
702 (SIGKILL candidates) go into a min-heap keyed by urgency score – most
703 urgent processed first."
704
705 -----
706 Q. Implement a frequency map and find top-k frequent elements.
707 -----
708 vector<int> topKFrequent(vector<int>& nums, int k) {
709     unordered_map<int,int> freq;
710     for(int x : nums) freq[x]++;
711
712     // Min-heap of size k
713     priority_queue<pair<int,int>, vector<pair<int,int>>, greater<>> pq;
714     for(auto& [val, cnt] : freq) {

```

```
    pq.push({cnt, val});
```

```
    if(pq.size() > k) pq.pop(); // keep only top k
```

```
}
```

```
vector<int> res;
```

```
while(!pq.empty()) { res.push_back(pq.top().second); pq.pop(); }
```

```
return res;
```

```
}
```

Time: $O(n \log k)$. Better than $O(n \log n)$ full sort.

YOUR LINK: "Same pattern I'd use to find top-k most-frequently scanned file types in my DLP engine telemetry."

Q. What is `std::transform`, `std::accumulate`, `std::for_each`?

Give a real use case for each.

ANSWER:

// transform: apply function to each element, write to output

```
vector<string> paths = {"/etc/passwd", "/tmp/test", "/home/user"};
```

```
vector<int> lengths;
```

```
transform(paths.begin(), paths.end(), back_inserter(lengths),
```

```
    [](const string& s){ return s.size(); });
```

```
// lengths = {11, 9, 10}
```

// accumulate: fold/reduce

```
int total = accumulate(lengths.begin(), lengths.end(), 0);
```

```
// total = 30
```

// accumulate with custom op (e.g., concatenate):

```
string all = accumulate(paths.begin(), paths.end(), string(""),
```

```
    [](string a, const string& b){ return a + ":" + b; });
```

// for_each: side effects

```
for_each(paths.begin(), paths.end(),
```

```
    [](const string& p){ cout << p << "\n"; });
```

// vs range-for: `for_each` is composable, can be passed as argument.

Q. What is `std::partition` and `std::stable_partition`?

Write a one-liner to move all negative numbers before positives.

ANSWER:

```
vector<int> v = {3, -1, 4, -2, 5, -6};
```

// unstable (may reorder within groups, $O(n)$ swaps):

```
auto mid = partition(v.begin(), v.end(), [](int x){ return x < 0; });
```

```
// [-1,-2,-6, 3,4,5] – order within groups not preserved
```

// stable (preserves relative order, $O(n \log n)$):

```
stable_partition(v.begin(), v.end(), [](int x){ return x < 0; });
```

```
766 // [-1,-2,-6, 3,4,5] – relative order preserved
767
768 // mid points to first positive element – useful for further processing.
769
770 YOUR LINK: "I use partition in my file scanner to separate blocked
771 files from allowed files in a single pass over the scan results vector."
772
773 -----
774 Q. What is std::move_iterator? When would you use it?
775 -----
776 ANSWER: Wraps an iterator so dereferencing it produces an rvalue.
777 Lets you "move" elements from a container instead of copying.
778
779 vector<string> src = {"alpha", "beta", "gamma"};
780 vector<string> dst;
781 // Copy (src unchanged):
782 copy(src.begin(), src.end(), back_inserter(dst));
783 // Move (src elements now in valid-but-unspecified state):
784 move(src.begin(), src.end(), back_inserter(dst)); // std::move (algorithm)
785 // OR:
786 dst.insert(dst.end(),
787     make_move_iterator(src.begin()),
788     make_move_iterator(src.end()));
789
790 // After move: src[0] etc. are valid but likely empty strings.
791
792 USE WHEN: Moving large strings/objects between containers without copy.
793
794 -----
795 Q. What is the difference between std::copy and std::move (algorithm)?
796 Are they related to std::move (utility)?
797 -----
798 ANSWER: Three different things with similar names:
799
800 std::move (utility) – in <utility>: casts a value to rvalue reference.
801 NOT an algorithm, does NOT move data by itself.
802
803 std::move (algorithm) – in <algorithm>: like std::copy but moves
804 each element using move semantics instead of copying.
805
806 std::copy (algorithm) – copies elements from source range to destination.
807
808 // Names are overloaded by the standard committee – common confusion.
809 // On an interview, clarify which one you mean.
810
811
812 =====
813 SECTION 5 – TRICKY OUTPUT / TRAP QUESTIONS (Engineer Level)
814 =====
815
816 Q1. What is the output?
```



```

817     vector<int> v = {1, 2, 3};
818     for(auto x : v) v.push_back(x * 2);
819     cout << v.size();
820     ANSWER: UNDEFINED BEHAVIOR.
821     push_back may reallocate, invalidating the range-for's internal
822     iterators. Say "UB due to iterator invalidation during iteration."
823
824 Q2. What is the output?
825     vector<int> v(5, 0);
826     fill(v.begin(), v.end(), 7);
827     cout << v[3];
828     ANSWER: 7. fill sets all elements to 7.
829
830 Q3. What is the output?
831     string s = "abcde";
832     reverse(s.begin(), s.begin() + 3);
833     cout << s;
834     ANSWER: "cbaDE"? No – "cbade". reverse first 3 chars: "abc" → "cba".
835     Rest unchanged. Result: "cbade".
836
837 Q4. What does this do?
838     vector<int> v = {5,3,1,4,2};
839     nth_element(v.begin(), v.begin()+2, v.end());
840     cout << v[2];
841     ANSWER: 3. nth_element partially sorts so that v[2] contains the
842     element that would be at index 2 in a sorted array. Elements before
843     index 2 are <= v[2], elements after are >= v[2], but not fully sorted.
844     O(n) average – faster than full sort when you only need the kth element.
845
846 Q5. What is the output?
847     map<int,int> m;
848     m[5]; m[3]; m[7];
849     for(auto& [k,v] : m) cout << k << " ";
850     ANSWER: 3 5 7 (map is always sorted by key)
851
852 Q6. What is the output?
853     unordered_map<string,int> um;
854     um["b"] = 2; um["a"] = 1; um["c"] = 3;
855     for(auto& [k,v] : um) cout << k << " ";
856     ANSWER: ORDER UNDEFINED. Could be "b a c" or "c b a" or any order.
857     This is a trap question – say "unordered_map has no guaranteed order."
858
859 Q7. What is wrong with this code?
860     vector<int> v = {1,2,3,4,5};
861     for(auto it=v.begin(); it!=v.end(); ++it)
862         if(*it % 2 == 0) v.erase(it);
863     ANSWER: UB – erase invalidates it. Use it = v.erase(it) instead
864     (erase returns next valid iterator). Or use erase-remove idiom.
865
866 Q8. What does std::unique do on this vector?
867     vector<int> v = {1,1,2,2,3,1,1};

```

```
868     auto it = unique(v.begin(), v.end());
869     v.erase(it, v.end());
870     cout << v.size();
871     ANSWER: 4. unique removes CONSECUTIVE duplicates only.
872     After: {1,2,3,1} – the second group of 1s at the end is kept
873     because it's not adjacent to the first group.
874     If you want all duplicates removed: sort first, then unique.
```

```
875
876
877 =====
```

878 SECTION 6 – 2 EXTRA SMART QUESTIONS TO ASK INTERVIEWER

879 (These are more technical than in the previous doc)

```
880 =====
```

881

882 QUESTION A:

883 "When you say Windows SDK and KMDF in the JD – is the existing
884 codebase using WDF object model with WDFDEVICE/WDFREQUEST, or
885 is it closer to raw WDM with IRPs directly? I'm ramping up on
886 Windows and want to know which abstraction layer to focus on."

887
888 WHY: Shows you already know the distinction between WDF (modern) and
889 WDM (low-level). Any Windows kernel engineer will immediately know
890 you've done your homework. Very few candidates ask this.

891

892 QUESTION B:

893 "For the DLP and EDR modules – are kernel components communicating
894 with user-space via IOCTL/DeviceIoControl, or do you use a filter
895 driver with the minifilter framework? I ask because my Linux equivalent
896 uses Unix sockets for IPC, and I want to map it correctly to Windows."

897
898 WHY: Shows you understand the Windows IPC model at driver level and
899 that you're already thinking about how to apply your existing knowledge.
900 This question will surprise most interviewers in a very good way.

```
901
902
903 =====
```

904 PRIORITY STUDY ORDER – TONIGHT (you have a few hours)

```
905 =====
```

906

907 HOUR 1:

- 908 - Puzzles 1, 2, 4, 7 (most commonly asked)
- 909 - Vector invalidation rules (will definitely be asked)
- 910 - string_view vs string

911

912 HOUR 2:

- 913 - Linked list: merge sorted, remove Nth from end, palindrome check
- 914 - Stack: MinStack, next greater element
- 915 - Vector: erase-remove idiom, emplace_back vs push_back

916

917 HOUR 3:

- 918 - STL: unordered_map internals, priority_queue min-heap

919 - String: find() npos trap, strstr, split
920 - Output traps: Section 5 all 8 questions
921
922 BEFORE SLEEP:
923 - Re-read your project explanations in your existing PDFs
924 - Link every answer back to something you actually built
925
926 =====
927 TOMORROW AT 2 PM – WALK IN LIKE YOU BUILT THE PRODUCT.
928 BECAUSE YOU KIND OF ALREADY DID.
929 =====
930